

Evaluating Linux Pipes in a Vacuum

Patrick DeCabooter

1 Introduction

Inter-process communication (IPC) is a critical trade-off of virtual memory systems. Before we had virtual memory, IPC was as simple as passing the pointer to the data between the two processes. However, when each process is sand-boxed in its own virtual memory environment, the process sending the data has no way to reason about the addresses of the process receiving the data, and vice-versa. Unfortunately for IPC, virtual memory has countless benefits and is here to stay.

Enter pipes, the first and oldest method of IPC chosen by the Linux kernel. Linux pipes create one-way channels between two processes. They contain two file descriptors, one to write to, and one to read from. In between writes and reads, the transfer payload is stored in a kernel-space buffer.

In this report we will:

- (1) Evaluate the precision of two Linux timers that will be used for measurement.
- (2) Measure the minimum latency of pipes when sending various payload sizes.
- (3) Measure the average throughput of pipes when continuously sending large payloads.

2 Linux Timers

We tested two types of timers for our experiments. The first timer, provided by the header `time.h`, is `clock_gettime`. We chose `CLOCK_MONOTONIC` as the clock to use for `clock_gettime` due to it having guaranteed monotonicity and being system-wide. `clock_gettime` provides nanosecond precision. The second timer, provided by the header `sys/time.h`, is `gettimeofday`. `gettimeofday` finds the time

by measuring the clock cycles since the last kernel epoch. `gettimeofday` provides microsecond precision. `gettimeofday` is a much older clock. By the time `cpu-cycles` were able to measure nanoseconds, `gettimeofday`'s interface was already locked-in to microseconds. `clock_gettime` was introduced as an api to replace `gettimeofday`. In order to have similar units, we scaled `gettimeofday` up to nanoseconds instead of scaling `clock_gettime` down to microseconds. We chose to do it this way since it would take advantage of `clock_gettime`'s increased accuracy and give us more intuitive numbers for figures.

3 Linux Pipes

When data moves between processes in a pipe, it is not as simple as the other process reading it out of virtual memory. First, the data must be sent by the `write` system call that context-switches into kernel mode and copies the data into a kernel-space buffer. Next, the reader must call the `read` system call that once again context-switches into kernel mode and copies the data in the kernel-space buffer back into a buffer in the user-space of that process. Finally, often times the reader will be programmed to send back an acknowledgement byte to the writer so it knows the message has been received. Since pipes are one-directional, this acknowledgement is done on a separate pipe.

We did not test pipes with any flags enabled or changes to buffer sizes. This means that the kernel is using the default pipe buffer size of 64KiB. This also means that `write` and `read` operated in blocking mode. Therefore, `write` will block until all bytes of the payload are written and `read` will block until it is able to read data from the pipe.

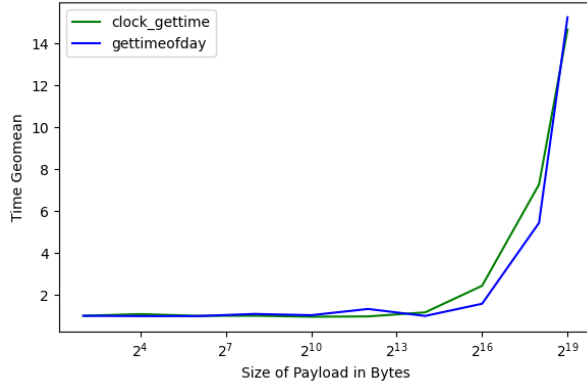


Figure 1: Latency of pipe across different payload sizes. The x-axis shows payload size in bytes on a log base-2 scale. The y-axis shows the geometric mean relative to the latency of the smallest payload size of 4 bytes.

4 Evaluation

Clock Precision

To evaluate the accuracy of these two timers, we decided to measure accuracy against `sleep` from the header `unistd.h` and measure the smallest measurable time. To measure against `sleep`, we started the timer, called `sleep(5)`, then measured the final time to get the difference. We did this five times in a loop for each timer, and took the minimum result. To measure the smallest measurable time, we had to compromise to instead find the smallest measurable time for `gettimeofday` since it measures in microseconds instead of nanoseconds. To make a one microsecond delay, we summed the numbers from 0 to 99 in a loop. We did this this 10 times and took the minimum result. We used minimums as our metric for both tests since we wanted to find the highest fidelity possible if our machine was running perfectly.

Pipe Latency

We tested pipe latency under 10 different payload sizes: 4, 16, 64, 256, 1K, 4K, 16K, 64K, 256K, and

512K bytes. Since synchronizing clocks between processes is unwieldy, we measured the time from payload send to acknowledgement acquisition. To construct our experiment, we had two processes. The parent process forked itself at the start of execution. The child enters a loop where it waits for the parent to send data then acknowledges the request. The parent sends each payload size 10 times. Both experiments using the `clock_gettime` and `gettimeofday` timers.

As seen in Figure 1, until the payload size reaches 16KiB, the latency appears to be constant. At payload sizes above 64KiB, the latency begins to grow linearly. We suspect that this scaling is due to these larger payload sizes hitting the default pipe buffer size. Once the kernel's pipe buffer is full, the `write` system call must block until it is emptied enough for more data to flow. To further dig into this hypothesis, we used the `strace` utility to measure the time spent in `wait4` by the parent process. First we tested together the payload sizes below 64KiB, then we tested together the payload sizes above 64KiB. Using the small payload sizes, `wait4` took only 15% of execution time. Using the large payload sizes, `wait4` took 35% of execution time. This backs up our hypothesis that smaller payload sizes spend less time waiting. However, given more time, testing different kernel buffer sizes would be able to further elucidate the latency relationship between kernel buffer sizes and payload sizes.

Pipe Throughput

5 Conclusion

Inter-process Communication (IPC) presents many problems and pipes are just one way to solve them. Despite their old age, pipes can still be seen in systems today. Because they are so common, it is important that we understand their performance properties.

Computer Specification

Component	Specification
Computer Model	HP - Elite Dragonfly G4
Architecture	x86-64
Intel Core i7-1355U	10 Cores / 12 Logical Processors
RAM	32 GiB / 4800 MT/s
NVMe SSD	1 TB
Virtualization	WSL2
OS	Ubuntu 24.04.3 LTS
Kernel	Linux 6.6.87.2-microsoft-standard-WSL2
Compiler	gcc version 13.3.0